

Generative AI, LLM & RAG Testing

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This syllabus lists every module of the course in teaching order, with the topics covered in each. It is generated from the course page, so the two cannot disagree.

Module 1: AI Application Architecture for Testers

Schedule: Week 1

What you are actually testing, described at the level a tester needs to design tests against it. The prompt-to-model-to-response path, system prompts versus user prompts, tokens and context windows, temperature and sampling, structured output, and tool or function calling in outline.

The module closes on non-determinism and its consequence, which is the thesis of the course: exact expected-value assertions do not work here, and understanding why they fail is what makes everything in the following eleven modules make sense rather than feel arbitrary.

Topics

- Generative AI concepts for testers
- LLM application architecture end to end
- Prompt → model → response
- System prompts versus user prompts
- Tokens and context windows
- Temperature and sampling
- Structured output and schema-constrained generation
- Tool and function calling in outline
- Model APIs and what a test actually calls
- Non-determinism, and why assertEquals fails

Module 2: Quality Dimensions for LLM Responses

Schedule: Week 1

If you cannot assert equality, you have to say what "good" means — precisely enough that two people grading the same answer agree. This module builds that vocabulary: correctness, relevance, completeness, conciseness, coherence, consistency, instruction following, format compliance, tone, groundedness, faithfulness, safety and refusal correctness.

Each is defined operationally, with the boundaries drawn where they blur — groundedness against faithfulness, correctness against relevance — because a rubric whose dimensions overlap produces scores nobody can act on.

Topics

- Correctness, relevance and completeness
- Conciseness and coherence
- Consistency across repeated runs
- Instruction following as a measurable property
- Format compliance and schema validity
- Tone and style adherence
- Groundedness versus faithfulness
- Safety and harm categories
- Refusal correctness — refusing the right things, not everything
- Writing a dimension definition two graders will agree on

Module 3: LLM Test Design

Schedule: Week 2

Test design for probabilistic systems. The input space is natural language, so the equivalent of boundary value analysis is prompt variation: paraphrases, typographical errors, ambiguity, conflicting instructions, missing context, and context long enough to push earlier instructions out of the window.

Multi-turn conversations, persona and language variation, and adversarial cases follow. The output is a regression suite structured so it can be re-run against a new model or a changed prompt, which is the event this whole discipline exists to survive.

Topics

- Functional test scenarios for an AI feature
- Prompt variation as the boundary-analysis equivalent
- Paraphrasing and typographical robustness
- Ambiguous prompts and conflicting instructions
- Long context and missing context
- Multi-turn conversation tests
- Persona and language variation
- Edge cases and adversarial cases
- Structuring a regression suite for model and prompt changes

Module 4: Hallucination and Grounding Tests

Schedule: Week 2

The defect class that made AI testing a job. Hallucination defined precisely — unsupported claims, fabricated facts, fabricated citations — and distinguished from ordinary incorrectness, because the mitigations differ.

You test knowledge-boundary behaviour, which is where good systems separate from bad ones: a system that says it does not know when it does not know is often more valuable than one that is right more often and confident always.

Source-grounded answers, citation validation and ground-truth comparison give you the mechanics for catching it.

Topics

- What hallucination is, and what it is not
- Unsupported claims and fabricated facts
- Fabricated citations and how to detect them
- Contradictory responses across runs
- Knowledge-boundary behaviour
- Testing for a correct "I don't know"
- Source-grounded answers
- Citation validation against the source
- Ground-truth comparison methods

Module 5: Evaluation Datasets

Schedule: Week 3

The golden dataset is the artefact this discipline runs on, and building a good one is most of the work. Reference answers, expected characteristics where a reference answer is impossible, rubrics, and the curation of edge cases that matter rather than edge cases that are easy to write.

Production-derived cases are treated as the highest-value source — real user prompts contain phrasings nobody on the team would invent — alongside the privacy handling that requires. Dataset versioning and regression baselines close the module, because an evaluation you cannot compare to last week's is not a regression test.

Topics

- Test datasets versus golden datasets
- Reference answers and expected characteristics
- Writing rubrics that grade consistently
- Production-derived test cases, and the privacy handling
- Synthetic test-case generation
- Curating edge cases that matter
- Dataset versioning
- Train, dev and test separation in outline
- Regression baselines and what they must record

Module 6: Evaluation Methods

Schedule: Weeks 3–4

The methods themselves, cheapest and most reliable first. Deterministic assertions still apply to more than testers expect — schema validity, required keywords, forbidden content, length and format constraints — and every check you can make deterministic is one you do not have to judge.

Then semantic similarity and embedding comparison, human evaluation with rubrics, and LLM-as-a-Judge: pairwise, reference-based and reference-free. Judge bias gets its own treatment — position bias, verbosity bias, self-preference — because a judge you have not validated is an unvalidated test tool making release decisions.

Topics

- Deterministic assertions that still apply
- Regex, rule-based and constraint checks
- Schema validation of structured output
- Semantic similarity and embedding comparison
- Human evaluation and rubric scoring
- LLM-as-a-Judge: pairwise, reference-based, reference-free
- Judge bias — position, verbosity, self-preference
- Validating a judge against human grades
- Evaluation repeatability and run counts

Module 7: Python-Based AI Test Automation

Schedule: Week 4

Turning the methods into a suite that runs. Calling model APIs from tests, pytest structure for AI tests, parametrised prompt tests, and handling structured outputs.

The operational details that separate a demo from a working harness get proper time: retry policy for rate limits and transient failures, capturing model metadata so a result is reproducible, token and cost tracking, persisting results, and comparing against a stored baseline. CI execution is covered in concept here and built out in the capstone.

Topics

- Calling model APIs from tests
- pytest structure for evaluation suites
- Parametrised prompt tests
- Handling and validating structured outputs
- Retry policy for rate limits and transient errors
- Capturing model, version and parameters with every result
- Token and cost tracking per test
- Persisting results for comparison
- Baseline comparison and diff reporting
- Test reports a release manager can read

Module 8: RAG Fundamentals for Testers

Schedule: Week 5

Retrieval-Augmented Generation explained as a pipeline with failure points, which is the useful framing for a tester. Why RAG exists, then documents, chunking, embeddings, vector stores, similarity search, hybrid retrieval, reranking, context assembly, generation and citation.

The module ends by mapping every stage to how it fails — retrieved nothing, retrieved the wrong thing, retrieved the right thing and ignored it, retrieved a stale version, retrieved another tenant's document. That map is what the next module tests against.

Topics

- Why RAG exists and what it fixes
- Documents, chunking and chunk boundaries
- Embeddings and vector stores
- Similarity search and hybrid retrieval
- Reranking in concept
- Context assembly and the prompt that results
- Generation and citation
- RAG failure modes, stage by stage

Module 9: RAG Evaluation

Schedule: Week 5

Two systems, evaluated separately and together, because a RAG answer can be wrong for opposite reasons. Retrieval metrics — relevance, context precision, context recall, coverage — tell you whether the right material was found. Generation metrics — answer relevance, faithfulness, groundedness, correctness, citation correctness — tell you whether it was used.

The adversarial cases follow: the missing document, the contradictory document, the stale document, and access-control tests for whether one tenant's query can surface another tenant's data. That last one is the finding that stops releases.

Topics

- Retrieval relevance and context precision
- Context recall and coverage
- Answer relevance and correctness
- Faithfulness and groundedness of the answer
- Citation correctness against retrieved context
- Missing-document behaviour
- Contradictory-document behaviour
- Stale-document and freshness tests
- Access-control and tenant-isolation tests
- RAG regression across index rebuilds

Module 10: AI Safety and Security Testing

Schedule: Week 6

The module that answers the question a customer's security review asks first. Prompt injection — direct, and the harder indirect case where the payload arrives inside a retrieved document — jailbreak patterns, and unsafe output.

Then the leakage categories: sensitive data, PII, secrets, and system-prompt disclosure. Policy adherence, role and permission boundaries and data isolation follow, with red-team thinking taught as a habit and safety regression suites as the deliverable, since a safety fix that is not regression-tested does not stay fixed.

Topics

- Direct prompt injection

- Indirect injection through retrieved content
- Jailbreak patterns and why they keep working
- Unsafe output categories
- Sensitive data and PII leakage
- Secret leakage
- System-prompt disclosure
- Policy adherence testing
- Role, permission and data-isolation boundaries
- Red-team thinking as a testing habit
- Building a safety regression suite

Module 11: Performance, Reliability and Cost

Schedule: Week 6

Non-functional testing for AI systems, where cost is a first-class metric in a way it never was for a web application. End-to-end latency, time to first token and total generation time; token usage and cost per request and per test.

Reliability covers rate limits, retries, timeouts, API errors, concurrent load, model availability and fallback behaviour — a fallback to a cheaper model that nobody tested is a silent quality regression. The module closes on the quality-versus-cost trade-off, which is the conversation these numbers are collected for.

Topics

- End-to-end latency and time to first token
- Total generation time under load
- Token usage measurement
- Cost per request and per test run
- Rate limits and retry behaviour
- Timeouts and API error handling
- Concurrent users and throughput
- Model availability and fallback testing
- Quality versus cost trade-offs

Module 12: Evaluation Frameworks and the Capstone

Schedule: Weeks 7–8

Concepts first, tools second, and deliberately so — the evaluation-framework landscape turns over faster than any curriculum, and a course built on one vendor's API ages badly. You write custom Python evaluators first, then look at DeepEval, RAGAS, Promptfoo and LangSmith as examples of the same patterns packaged differently.

CI-based evaluation gates, model and prompt comparison reporting, and then the capstone: a working evaluation harness for an LLM feature and a RAG system, with golden datasets, safety tests, baselines and a report a release manager can act on.

Topics

- Writing custom Python evaluators
- DeepEval concepts
- RAGAS concepts
- Promptfoo concepts
- LangSmith evaluation concepts
- Open-source evaluation patterns
- CI-based evaluation gates and thresholds
- Model and prompt comparison reporting
- Capstone — golden dataset, evaluators, safety suite, baselines
- Presenting a quality report to a release decision

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.

ARCHER INFOTECH PUNE
archerinfotech.in